

CHAPTER 1 METHODOLOGY

1.1 Research Design

This study, “FeatureSAGE: Stable Attribute Group Editing With Integrated StyleFeatureEditing Encoder for Enhanced Few-Shot Image Generation,” adopts a quantitative experimental research design to evaluate improvements to the existing SAGE framework. In particular, the GAN inversion module of SAGE is modified by integrating the StyleFeatureEditor (SFE) inverter and its Feature Editor module. The modification is evaluated under a one-shot setting, where attribute manipulation is conditioned on a single reference image per test instance. Objective evaluation is performed using Fréchet Inception Distance (FID) and Learned Perceptual Image Patch Similarity (LPIPS), which measure distributional image quality and perceptual variation, respectively.

1.1.1 Experimental Setup

The experimental setup is designed to evaluate the effectiveness of the proposed FeatureSAGE framework in comparison to the existing SAGE method. The evaluation follows a quantitative experimental design that involves controlled changes to the inversion and feature-editing components and the measurement of model outputs using standardized metrics.

- **Independent Variable:** The main experimental manipulation is the GAN inversion method. The baseline SAGE configuration uses the original pixel2style2pixel (pSp) encoder, whereas the proposed FeatureSAGE configuration integrates the StyleFeatureEditor inverter and Feature Editor module.
- **Dependent Variables:** To objectively evaluate the effectiveness of the GAN inversion and editing pipeline, the following metrics are employed:

- Reconstruction quality and generated-image distribution alignment, measured using Fréchet Inception Distance (FID).
- Perceptual similarity or perceptual diversity, depending on the evaluation context, measured using Learned Perceptual Image Patch Similarity (LPIPS).

- **Controlled Variables**

1. **Generator Architecture:** A fixed, pretrained StyleGAN2 generator is used in both configurations for consistent image synthesis capability.
2. **Training Dataset:** Both configurations use the same FUNIT Animal Faces and 102 Flowers datasets with the same preprocessing and seen/unseen class splits.
3. **Evaluation Protocol:** The same unseen categories, one-shot support samples, query sets, and evaluation procedures are used for both configurations.
4. **Loss Functions:** Reconstruction and editing losses are applied consistently within the corresponding training phases.
5. **Optimization Settings:** Both configurations use the Adam optimizer and the fixed learning-rate settings specified in the training strategy.
6. **Evaluation Metrics:** FID and LPIPS are computed using the same implementations and parameters.
7. **Hardware and Software Environment:** All experiments are conducted using the same GPU-enabled PyTorch environment to avoid performance inconsistencies.

- **Experimental Conditions**

Two experimental configurations are used:

- SAGE (Baseline): Employs the original pSp encoder with StyleGAN2.
- FeatureSAGE (Proposed): Integrates the StyleFeatureEditor inverter and Feature Editor.

1.1.2 Hardware and System Requirements

All experiments were conducted on a virtual machine instance with GPU pass-through hosted on institutional cloud infrastructure. The same hardware and software configuration was used for the baseline and proposed models so that performance differences could be attributed to the inversion and feature-editing components rather than to changes in the execution environment. Table 1.1 summarizes the system requirements reported for the implementation.

Component	Specification
CPU	QEMU Virtual CPU version 2.5+, 2 processors at 2.19 GHz
RAM	32.0 GB
GPU	NVIDIA GRID V100D-32A, virtualized Tesla V100 with 16 GB HBM2
Storage	Not specified in the source implementation notes; storage is required for the datasets, model checkpoints, and generated outputs.
Software and frameworks	PyTorch environment; pretrained StyleGAN2 generator; pSp encoder for the baseline and editing-pair generation; StyleFeatureEditor inverter and Feature Editor; MTCNN for animal-face cropping and alignment; Adam optimizer; mixed precision for Feature Editor training; FID and LPIPS metric implementations.

Table 1.1 Hardware and system requirements used in the experimental setup.

The GPU specification is important because FeatureSAGE performs multiple generator passes and feature-tensor operations during inversion and editing. The use of mixed precision in the Feature Editor phase reduces memory pressure on the 16 GB V100 vGPU while preserving the same model configuration and reported evaluation protocol.

1.1.3 Few-Shot Experimental Setting

In this study, the term *few-shot* refers to the number of reference images available per target class used to construct class-level latent representations for attribute group editing and downstream evaluation. A **one-shot** setting utilizes a single reference image per target class, representing an extreme data-scarce scenario in which attribute editing and inversion must be performed with minimal visual information. This one-shot configuration is applied consistently across all datasets and experimental conditions for both the baseline SAGE and the proposed FeatureSAGE framework. By explicitly controlling the evaluation to a single reference image per class, the experimental protocol enables a systematic assessment of FeatureSAGE’s robustness, stability, and generalization capability under severe data limitations. The few-shot constraint is applied during test-time adaptation and evaluation on unseen classes, while the StyleGAN2 generator and attribute directions are learned from seen-category data.

1.2 Data Preprocessing

1.2.1 Data Sources

This research uses publicly available datasets containing structured visual categories for few-shot image generation and attribute editing. The datasets are selected because they contain clearly grouped classes and visual variation that can be used to evaluate whether generated images preserve category-level structure while allowing appearance changes. Only a limited number of representative samples per class are used during few-shot evaluation to reproduce the data-scarce setting of the study. The datasets used are:

1. FUNIT Animal Faces Dataset

Figure 1.1 shows representative samples from the FUNIT Animal Faces dataset. The dataset is composed of 117,484 animal face images at 512×512 resolution and is organized into 149 classes, where each class corresponds to one animal species. Each



Figure 1.1 Sample images from the FUNIT Animal Faces dataset. Source: (?)

domain includes a diverse range of species and breeds, ensuring variation in appearance while maintaining consistent facial alignment to support generative model training. The images were curated from openly available sources and standardized in resolution and alignment, with the eyes centrally positioned to provide consistency for learning. A subset of the images is reserved as a test set, with the remainder used for training. The diversity and quality of the FUNIT Animal Faces dataset make it a strong benchmark for generative and attribute-editing models in few-shot settings (?).

2. 102 Flowers Dataset



Figure 1.2 Sample images from the 102 Flowers dataset. Source: (?)

Figure 1.2 shows sample images from the 102 Flowers dataset, which comprises 102 flower classes found in the United Kingdom. Each class contains between 40 and 258 images. The dataset includes variation in flower size, pose, and lighting conditions, and it contains both large intra-class differences and visually similar categories. These characteristics make the dataset suitable for evaluating fine-grained category preservation and appearance variation. The raw images have varying dimensions; for example, one raw image from the dataset measures 500×667 pixels (?).

1.2.2 Data Preprocessing Steps

The same preprocessing pipeline is applied across the baseline SAGE and proposed FeatureSAGE configurations:

- **Resizing and normalization:** Images are resized according to the StyleGAN2 configuration used in the experiment and are normalized to the range $[-1, 1]$ before inversion or synthesis. The quantitative results reported in Chapter 4 are evaluated at 1024×1024 resolution.
- **Filtering:** Extremely low-quality or mislabeled images are removed before training and evaluation. For the FUNIT Animal Faces dataset, MTCNN is used to crop and horizontally align animal faces so that the eyes are leveled.
- **Train-test splitting:** The datasets are partitioned into seen categories for model fitting and unseen categories for few-shot testing:
 - **Animal Faces:** Out of the total 149 categories, 119 are used as seen classes for training, while the remaining 30 serve as unseen classes for evaluation.
 - **Flowers:** Out of the total 102 categories, 85 are assigned to the seen training split and 17 are reserved for the unseen test split.
- **Augmentation:** No augmentation is used beyond centering and flipping, since the study focuses on evaluating the same domains under controlled few-shot conditions.

1.3 Model Architecture

This study proposes a modified SAGE framework named **FeatureSAGE**. The architecture combines a pretrained StyleGAN2 generator, the StyleFeatureEditor inverter and Feature Editor, and SAGE’s attribute group editing mechanism. The purpose of the modification is to improve the inversion stage of SAGE by representing each input image with both a layer-wise latent

code and an intermediate feature tensor. This design follows the observation of ? that stronger inversion can further improve the potential of SAGE for generative augmentation, and it follows ? in using both W^+ latents and feature latents for detail-preserving editing.

The main components of FeatureSAGE are:

1. **StyleGAN2 generator:** The pretrained StyleGAN2 generator synthesizes images from latent codes. In FeatureSAGE, the generator receives the edited W^+ latent code together with an edited intermediate feature tensor produced by the Feature Editor.
2. **StyleFeatureEditor inverter:** The inverter replaces the original pSp inversion module used in SAGE. For each input image, it predicts a W^+ latent code and an intermediate feature tensor. The W^+ code captures layer-wise style information, while the feature tensor carries spatial details that are difficult to preserve using latent codes alone.
3. **Feature Editor:** The Feature Editor modifies the extracted feature tensor according to the latent-space edit. This allows the decoder path of the generator to synthesize images whose semantic edit is controlled by the latent code while finer details are supported by feature-level information.
4. **SAGE attribute group editing:** The attribute factorization, class embedding mapping, and adaptive editing blocks are retained from SAGE. These blocks estimate category-relevant embeddings and category-irrelevant directions so that unseen classes can be edited under a one-shot setting.

1.3.1 Framework Overview

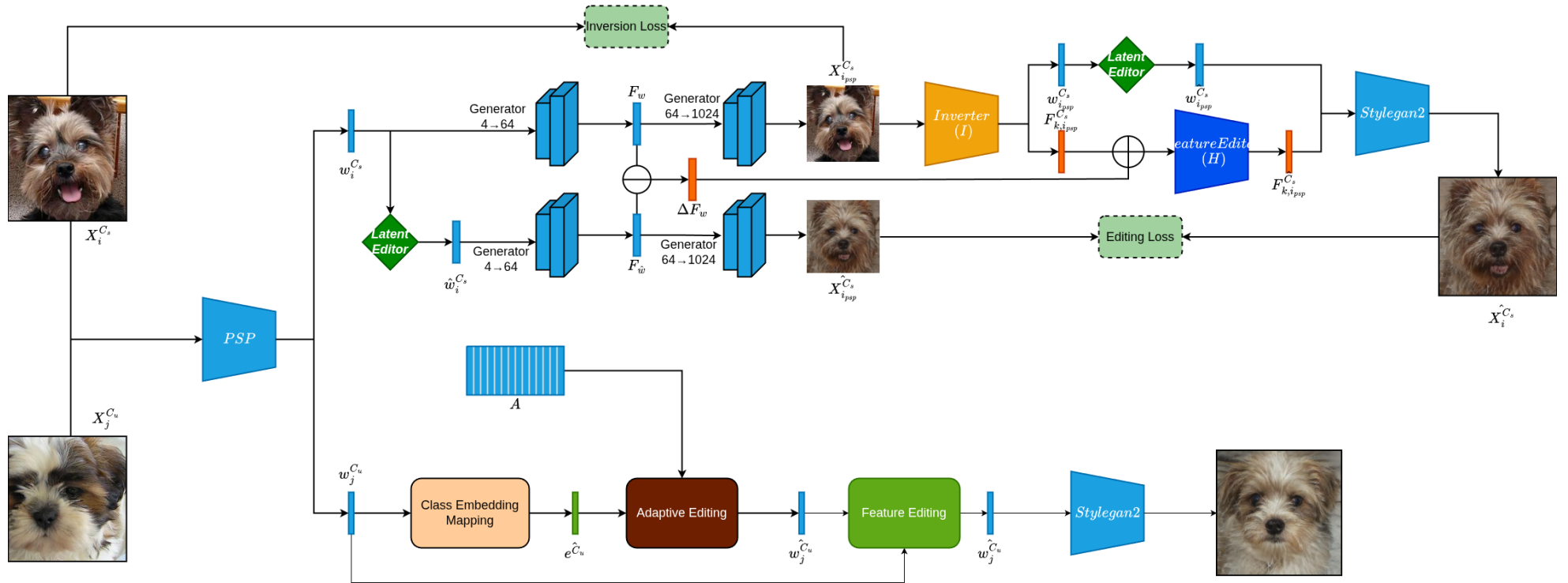


Figure 1.3 FeatureSAGE framework

Figure 1.3 shows the overall FeatureSAGE framework. The training set for seen categories and the testing set for unseen categories are represented in Equation 1:

$$D_{\text{train}} = \{x_i^{C_s}\}_{i=1}^{N_s}, \quad D_{\text{test}} = \{x_i^{C_u}\}_{i=1}^{N_u}, \quad (1)$$

where $x_i^{C_s}$ denotes an image from a seen category C_s , $x_i^{C_u}$ denotes an image from an unseen category C_u , and N_s and N_u denote the number of samples used from the seen and unseen splits, respectively. Equation 1 provides the notation used throughout the methodology for separating classes used in training from classes used during one-shot evaluation.

1.3.2 Feature Extraction and Latent-Space Representation

Feature extraction occurs before the generator synthesizes an output image. In the baseline SAGE configuration, the pSp encoder maps an input image into the extended StyleGAN2 latent space, as shown in Equation 2:

$$PSP(x_i^{C_s}) = w_i^{C_s}, \quad w_i^{C_s} \in W^+. \quad (2)$$

In Equation 2, $PSP(\cdot)$ is the pSp encoder and $w_i^{C_s}$ is the predicted W^+ latent code for a seen-category image. The W^+ space is important because it contains separate style vectors for the generator layers, allowing the model to represent coarse-to-fine visual attributes.

In FeatureSAGE, the StyleFeatureEditor inverter extracts two representations from an image, as defined in Equation 3:

$$I(x_i^C) = (w_i^C, F_{k,i}^C), \quad w_i^C \in W^+. \quad (3)$$

Here, $I(\cdot)$ is the StyleFeatureEditor inverter, w_i^C is the layer-wise latent code, and $F_{k,i}^C$ is the intermediate feature tensor extracted at generator layer k . The document uses the ninth StyleGAN feature layer for feature injection. The latent code represents semantic and style-level information, while the feature tensor represents spatial detail and texture

information. These two extracted features are significant because the latent code controls the edit direction, whereas the feature tensor helps preserve local details during synthesis.

The generator can also expose intermediate features for an original and edited latent code. Equation 4 defines these generator-side features:

$$(x_{i,\text{psp}}^{C_s}, F_w) = G(w_i^{C_s}), \quad (\hat{x}_{i,\text{psp}}^{C_s}, F_{\hat{w}}) = G(\hat{w}_i^{C_s}). \quad (4)$$

In Equation 4, $G(\cdot)$ is the StyleGAN2 generator, F_w is the intermediate feature tensor associated with the original latent code, and $F_{\hat{w}}$ is the feature tensor associated with the edited latent code. These feature tensors are later used to compute the feature-space change required by the Feature Editor.

The latent space in FeatureSAGE therefore contains three related types of information: the W^+ latent code w , the category-level class embedding e^C , and the intermediate feature tensor F_k . The W^+ code encodes layer-wise style information; the class embedding stores category-relevant information through an average latent representation; and F_k stores detail-rich spatial information used by the decoder block. Their combined use addresses the fidelity-editability trade-off identified in the literature without changing the reported experimental claims.

1.3.3 Attribute Factorization

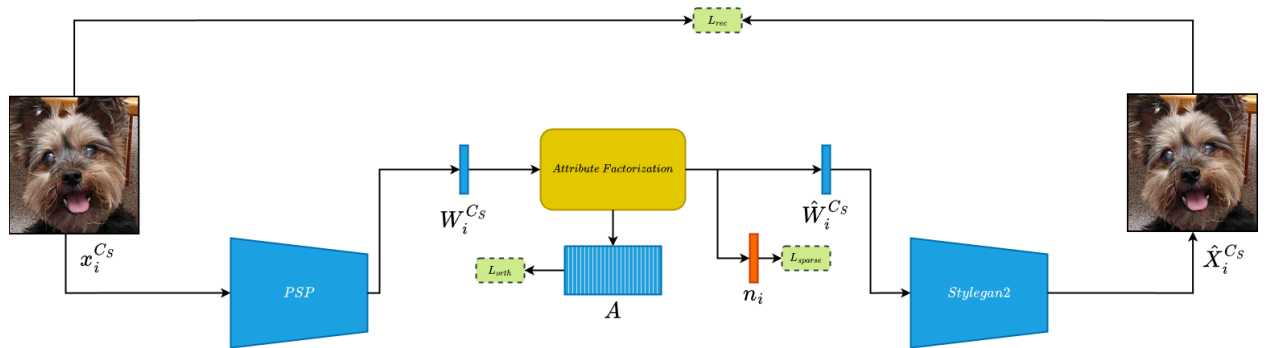


Figure 1.4 Attribute Factorization Training Framework (?)

To enable disentangled editing, FeatureSAGE retains the SAGE attribute factorization

block. The block learns category-irrelevant directions by comparing the latent code of a sample with the mean latent code of its category. The class embedding for a category is computed as the mean of its latent codes, as shown in Equation 5:

$$e^C = \frac{1}{N_C} \sum_{i=1}^{N_C} w_i^C, \quad (5)$$

where e^C is the class embedding for category C , N_C is the number of latent codes available for the category, and w_i^C is the latent code of the i th image. Equation 5 provides the category-relevant center used to separate identity or class-level information from editable appearance variation.

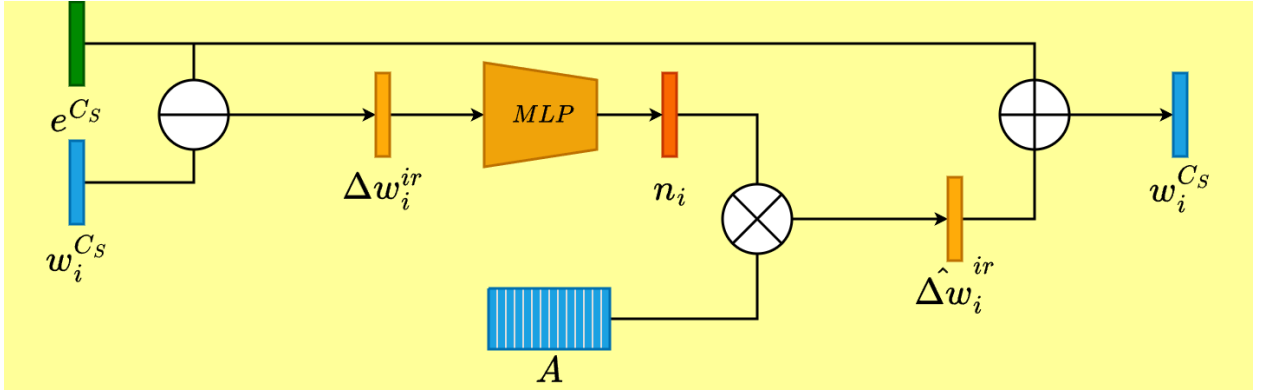


Figure 1.5 Attribute Factorization Block, derived from (?)

In the Attribute Factorization block shown in Figure 1.5, the category-irrelevant residual is computed in Equation 6:

$$\Delta w_i^{ir} = w_i^{Cs} - e^{Cs}. \quad (6)$$

In Equation 6, Δw_i^{ir} is the latent residual that remains after subtracting the class embedding e^{Cs} from the sample latent code w_i^{Cs} . The purpose of this residual is to isolate appearance attributes that affect how an object looks without changing its class membership, such as orientation, color, background, texture, and expression.

The residual is then reconstructed using a learned sparse dictionary, as shown in Equation 7:

$$\hat{\Delta w}_i^{ir} = An_i. \quad (7)$$

In Equation 7, A is the global dictionary of category-irrelevant directions and n_i is the sparse coefficient vector predicted for the image. The dictionary basis follows sparse dictionary learning: each image is represented using only a small number of active directions so that individual directions remain more semantically interpretable.

Category-irrelevant editing is defined in Equation 8. The edit changes appearance while preserving the category:

$$edit_{ir}(G(W_i^{C_s})) = G(W_i^{C_s} + \alpha \Delta W^{ir}, H(F_{k,i}, \alpha \Delta W^{ir})) = \hat{X}_i^{C_s}. \quad (8)$$

In Equation 8, $edit_{ir}(\cdot)$ denotes category-irrelevant editing, α controls the edit intensity, ΔW^{ir} is the category-irrelevant direction, $H(\cdot)$ is the Feature Editor, and $\hat{X}_i^{C_s}$ is the edited image. The theoretical basis is the SAGE assumption that category-irrelevant directions can be learned from seen categories and transferred to unseen categories.

The sparse dictionary learning objective is stated in Equation 9:

$$\min_{n_i} \|n_i\|_0 \quad \text{s.t.} \quad \Delta W_i^{ir} = An_i. \quad (9)$$

In Equation 9, $\|n_i\|_0$ counts the number of nonzero entries in n_i . The purpose of the L_0 constraint is to encourage sparse use of the dictionary so that each generated edit is controlled by a limited number of directions.

In practice, SAGE optimizes this factorization with an encoder-decoder structure in which a Multi-Layer Perceptron (MLP) predicts the sparse representation, as shown in Equation 10:

$$n_i = MLP(\Delta W_i^{ir}). \quad (10)$$

Equation 10 defines how the residual latent vector is encoded into the sparse coefficient

vector used by the dictionary A .

1.3.4 Class Embedding Mapping

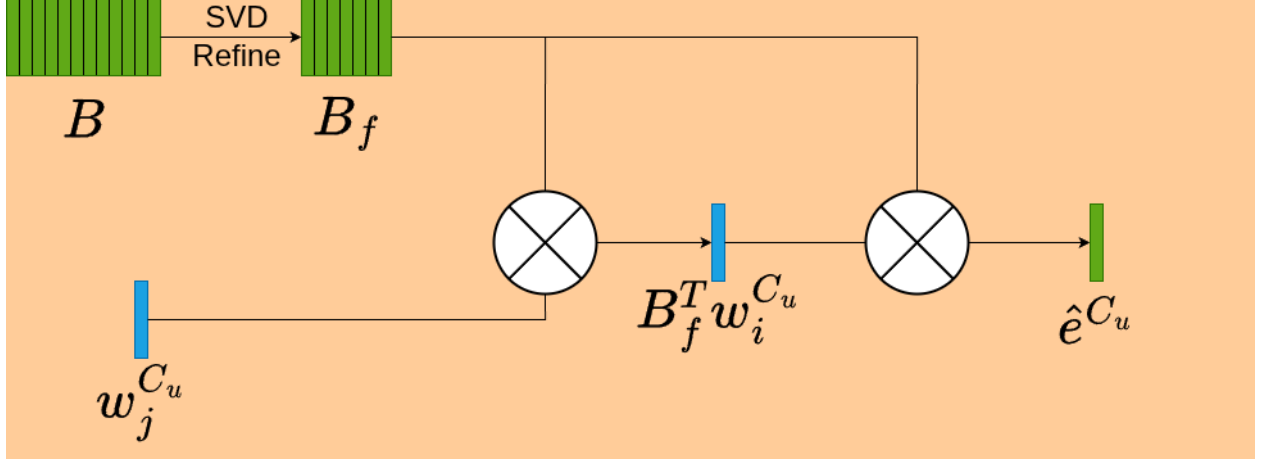


Figure 1.6 Class Embedding Block, adapted from (?)

Compared with Attribute Group Editing (AGE) (?), which randomly samples editing directions from A , SAGE uses class embedding mapping. The class embedding mapping block shown in Figure 1.6 identifies the most important directions in the latent-code space for each category group and uses those directions to guide subsequent edits. This part of the framework remains unchanged in FeatureSAGE.

The class embedding set for the seen categories is denoted as B . Since the embeddings are computed from average latent codes, B may still contain interference from category-irrelevant attributes. To filter the category-relevant subspace, singular value decomposition (SVD) is applied as shown in Equation 11:

$$B = U_B \Sigma_B V_B^T. \quad (11)$$

In Equation 11, B is the matrix of category-relevant class embeddings, U_B contains the left singular vectors, Σ_B contains singular values in descending order, and V_B^T is the transpose of the right singular-vector matrix. The top- t_B directions in U_B are selected as the final category-relevant attribute dictionary $B_f \in \mathbb{R}^{18 \times 512 \times t_B}$.

The class embedding of an unseen category is estimated using the filtered dictionary, as shown in Equation 12:

$$\hat{e}^{C_u} = \frac{1}{N_u} \sum_{i=1}^{N_u} B_f B_f^T W_i^{C_u}. \quad (12)$$

In Equation 12, $\hat{e}^{C_u}$ is the estimated embedding for unseen category C_u , N_u is the number of latent codes available for that unseen category, and $W_i^{C_u}$ is the latent code of the i th unseen-category sample. The projection $B_f B_f^T W_i^{C_u}$ retains category-relevant components while suppressing most category-irrelevant variation.

1.3.5 Adaptive Editing

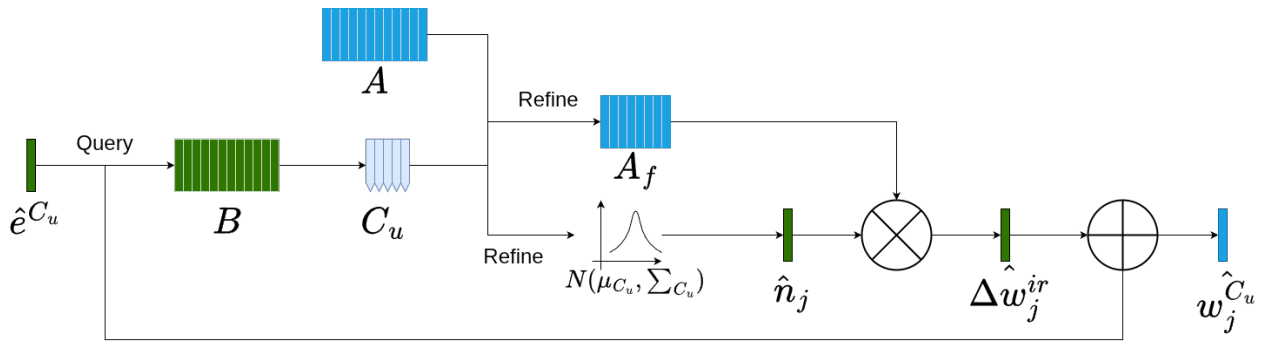


Figure 1.7 Adaptive Editing Block, from (?)

Figure 1.7 shows the adaptive editing block of FeatureSAGE. Category-irrelevant qualities typically have similar distributions in closely related categories but may differ across distant categories. Therefore, adaptive editing selects seen categories that are close to an unseen category in the class-embedding space and uses their common editing directions to guide generation.

To identify the category-irrelevant editing directions that best fit an unseen category C_u , the residual direction is back-projected into the sparse representation, as shown in Equation 13:

$$\hat{n}_i = A^\dagger \Delta W_i^{ir}. \quad (13)$$

In Equation 13, $A^\dagger$ denotes the pseudo-inverse of the dictionary A , ΔW_i^{ir} is the category-irrelevant residual, and $\hat{n}_i$ is the estimated sparse representation. The pseudo-inverse is used to express an observed residual in the coordinate system of the learned dictionary.

The mean absolute activation of sparse directions across the top- t_C similar seen categories is computed in Equation 14:

$$\overline{|n|}^{C_u} = \frac{1}{t_C} \sum_{t=1}^{t_C} \frac{1}{N_{S_t}} \sum_{i=1}^{N_{S_t}} |\hat{n}_i^{C_{S_t}}|. \quad (14)$$

In Equation 14, t_C is the number of similar seen categories, N_{S_t} is the number of samples in the t th selected seen category, and $\hat{n}_i^{C_{S_t}}$ is the sparse representation of the i th sample from that category. The result $\overline{|n|}^{C_u}$ measures which dictionary directions are frequently or strongly activated for categories similar to C_u . For each layer in W^+ , the top- t_A directions are selected from A to form the adaptive category-irrelevant dictionary $A_{C_u} \in \mathbb{R}^{18 \times 512 \times t_A}$.

SAGE assumes that the sparse representations for related categories follow a Gaussian distribution. This distribution is estimated in Equation 15:

$$\tilde{n}_j \sim \mathcal{N}(\mu_{C_u}, \Sigma_{C_u}). \quad (15)$$

In Equation 15, μ_{C_u} and Σ_{C_u} are the estimated mean and covariance of sparse codes for the selected categories, and $\tilde{n}_j$ is a sampled sparse code used to generate a new image. This sampling step provides controlled diversity for the unseen category.

The final adaptive generation equation is given in Equation 16:

$$X_j^{C_u} = G(\hat{e}^{C_u} + \alpha A_{C_u} \tilde{n}_j, H(F_{k,j}^{C_u}, \alpha A_{C_u} \tilde{n}_j)). \quad (16)$$

In Equation 16, $X_j^{C_u}$ is a generated image for unseen category C_u , $\hat{e}^{C_u}$ is the estimated class embedding, A_{C_u} is the adaptive dictionary, $\tilde{n}_j$ is the sampled sparse code, α controls the strength of the edit, and $H(\cdot)$ edits the feature tensor used by the generator. This equation summarizes how FeatureSAGE transforms one-shot class information and sampled attribute

variation into a generated image.

1.3.6 Feature Editing Decoder Block

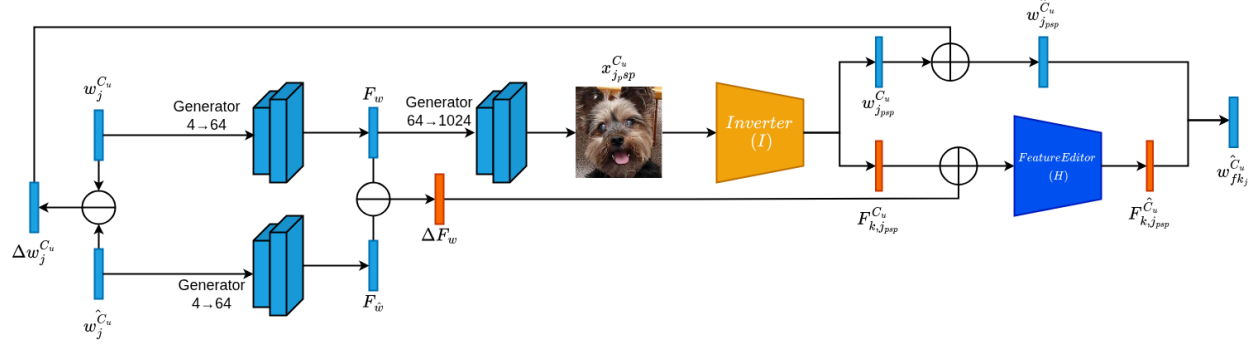


Figure 1.8 Feature Editing Decoder Block

Figure 1.8 shows the Feature Editing Decoder Block. The block receives the original latent representation, the edited latent representation produced by adaptive editing, and the intermediate feature tensor extracted by the StyleFeatureEditor inverter. Its role is to transform the input features into an edited feature tensor that can be injected into the ninth StyleGAN feature layer during image synthesis.

The latent-space difference used to describe the semantic edit is defined in Equation 17:

$$\Delta w_j^{C_u} = \hat{w}_j^{C_u} - w_j^{C_u}. \quad (17)$$

In Equation 17, $w_j^{C_u}$ is the original latent code and $\hat{w}_j^{C_u}$ is the edited latent code. This difference specifies the semantic movement applied in W^+ .

The feature-space difference used for feature injection is defined in Equation 18:

$$\Delta F_w = F_w - F_{\hat{w}}. \quad (18)$$

In Equation 18, F_w is the intermediate feature tensor produced from the original latent code and $F_{\hat{w}}$ is the corresponding tensor produced from the edited latent code. The decoder block uses this difference to determine how the original feature tensor should be modified to

match the target semantic edit.

The Feature Editor transformation is represented in Equation 19:

$$F_{k,j}^{\text{edit}} = H(F_{k,j}^{C_u}, \Delta F_w). \quad (19)$$

In Equation 19, $F_{k,j}^{C_u}$ is the feature tensor extracted by the inverter for the unseen-category input, ΔF_w is the feature difference from Equation 18, and $F_{k,j}^{\text{edit}}$ is the edited tensor injected into the generator. The decoder therefore modifies the feature-level representation while the edited latent code controls the global semantic direction. The output image is then synthesized by the generator using the edited latent code and the edited feature tensor, as shown in Equation 20:

$$\hat{X}_j^{C_u} = G(\hat{w}_j^{C_u}, F_{k,j}^{\text{edit}}). \quad (20)$$

Equation 20 completes the decoder pathway: input image features and latent edits are transformed into an edited high-resolution output.

1.3.7 Training Strategy

In the training stage, a StyleGAN2 generator is trained using images from the seen categories. The StyleFeatureEditor inverter and the pSp encoder are also trained or used according to their roles in the pipeline. To manage resource consumption, FeatureSAGE uses a two-phase strategy: attribute factorization training followed by Feature Editor training.

1.3.8 Phase 1: Attribute Factorization Training

Phase 1 trains the attribute factorization block for disentangled sampling of category-irrelevant directions. The following losses are applied to the attribute factorization block.

- **Sparsity Loss (L_{sparse}):** The sparsity loss encourages the learned attribute code to

activate only a small number of dictionary directions. Equation 21 defines the loss:

$$L_{\text{sparse}} = \|\sigma(\theta_0 n_i - \theta_1)\|_1. \quad (21)$$

In Equation 21, $\sigma(\cdot)$ denotes the sigmoid function, n_i is the sparse code, and θ_0 and θ_1 are hyperparameters controlling sparsity. The sigmoid compresses the MLP output and the L_1 norm approximates sparse behavior in a differentiable way.

- **Reconstruction Loss (L_{rec}):** The reconstruction loss anchors the edited output to the input image so that edits do not destroy the original content. Equation 22 defines this loss:

$$L_{\text{rec}} = \|G(e^{C_s} + An_i) - x_i^{C_s}\|_2. \quad (22)$$

In Equation 22, e^{C_s} is the class-center latent, An_i is the attribute offset, $G(\cdot)$ is the generator, and $x_i^{C_s}$ is the input image. The loss penalizes large differences between the generated reconstruction and the original image.

- **Orthogonality Loss (L_{orth}):** The orthogonality loss discourages category-irrelevant directions from overlapping with category-relevant directions. Equation 23 defines this loss:

$$L_{\text{orth}} = \|B^T A\|_F^2. \quad (23)$$

In Equation 23, A is the category-irrelevant dictionary, B is the category-relevant embedding basis, and $\|\cdot\|_F^2$ denotes the squared Frobenius norm. This term preserves class identity by reducing leakage between the category-relevant and category-irrelevant subspaces.

The overall attribute factorization loss is defined in Equation 24:

$$L_{\text{af}} = L_{\text{rec}} + \lambda_1 L_{\text{orth}} + \lambda_2 L_{\text{sparse}}. \quad (24)$$

In Equation 24, λ_1 and λ_2 are weights for the orthogonality and sparsity losses (?). Training uses the Adam optimizer with a fixed learning rate of 1×10^{-4} , $\lambda_1 = 5 \times 10^{-4}$, $\lambda_2 = 5 \times 10^{-3}$, $\theta_0 = 5 \times 10^{-1}$, and $\theta_1 = -1$.

1.3.9 Phase 2: Feature Editor Training

Phase 2 trains the Feature Editor H using editing pairs generated with a pretrained pSp encoder. Following ?, an input image X is encoded into w , edited by a sampled direction from the adaptive editing procedure, and paired with the corresponding edited image $\hat{X}$. This trains H to edit feature tensors according to the category-irrelevant attributes learned in Phase 1. To prevent the Feature Editor from relying only on synthetic editing pairs, it is also trained on the classical inversion task with $\Delta = 0$.

The editing loss is defined in Equation 25:

$$L_{\text{edit}}(X, \hat{X}) = \lambda_{l_2} \|X - \hat{X}\|_2 + \lambda_{\text{lpips}} \mathcal{L}_{\text{lpips}}(X, \hat{X}). \quad (25)$$

The inversion loss uses the same reconstruction and perceptual terms, as shown in Equation 26:

$$L_{\text{inv}}(X, \hat{X}) = \lambda_{l_2} \|X - \hat{X}\|_2 + \lambda_{\text{lpips}} \mathcal{L}_{\text{lpips}}(X, \hat{X}). \quad (26)$$

Equations 25 and 26 use L_2 reconstruction and LPIPS perceptual losses to preserve both pixel-level structure and perceptual similarity. The total Feature Editor loss is defined in Equation 27:

$$L_{\text{fe}} = L_{\text{edit}}(X, \hat{X}) + L_{\text{inv}}(X, \hat{X}). \quad (27)$$

In Equation 27, the editing term teaches H to respond to nonzero edit directions, while the inversion term with $\Delta = 0$ stabilizes feature reconstruction. The Feature Editor is trained using the Adam optimizer with a fixed learning rate of 1×10^{-4} for 15,000 iterations on a single

V100 16 GB vGPU. Mixed precision is enabled to improve memory efficiency because the phase requires multiple generator passes. The loss weights are $\lambda_{l_2} = 10$ and $\lambda_{\text{lips}} = 0.8$, with an additional LPIPS scaling factor of 0.5. The sparse dictionary A is initialized with a fixed length of 100.

1.4 Evaluation Protocol

1.4.1 Evaluation Metrics

The models are evaluated using quantitative metrics for generated-image distribution alignment and perceptual variation under a one-shot test-time setup. The same implementations and evaluation settings are used for AGE, SAGE, and FeatureSAGE.

1. Fréchet Inception Distance (FID)

Fréchet Inception Distance (FID) is used to evaluate how closely the distribution of generated images matches the distribution of real images (??). Instead of comparing images directly at the pixel level, FID computes statistics from a high-level feature space extracted using a pretrained Inception-v3 network. The metric models the real and generated image features as multivariate Gaussian distributions and computes the distance between them. Equation 28 defines the FID score:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}). \quad (28)$$

In Equation 28, μ_r and Σ_r are the mean and covariance of Inception features from real images; μ_g and Σ_g are the mean and covariance of Inception features from generated images; $\|\mu_r - \mu_g\|_2^2$ is the squared Euclidean distance between feature means; $\text{Tr}(\cdot)$ is the trace operator; and $(\Sigma_r \Sigma_g)^{1/2}$ is the matrix square root of the covariance product. The theoretical basis of FID is the Fréchet, or 2-Wasserstein, distance between two Gaussian feature distributions (?). Lower FID values indicate stronger distributional alignment

between generated and real images.

2. Learned Perceptual Image Patch Similarity (LPIPS)

Learned Perceptual Image Patch Similarity (LPIPS) compares images using deep feature activations from a pretrained neural network, such as VGG or AlexNet, rather than relying only on pixel-level differences (?). This makes LPIPS useful for evaluating perceptual similarity in reconstruction tasks and perceptual diversity when measured among generated outputs. Equation 29 defines the LPIPS score between images x and x' :

$$\text{LPIPS}(x, x') = \sum_l \frac{1}{H_l W_l} \sum_{h=1}^{H_l} \sum_{w=1}^{W_l} \|\omega_l \odot (\hat{y}_{hw}^l(x) - \hat{y}_{hw}^l(x'))\|_2^2. \quad (29)$$

In Equation 29, $\hat{y}_{hw}^l(x)$ is the unit-normalized feature vector of image x at layer l and spatial position (h, w) , ω_l is a learned channel-weight vector, and H_l and W_l are the height and width of the feature map at layer l . For reconstruction similarity, lower LPIPS values indicate closer perceptual similarity. In the results table, LPIPS is reported as a perceptual-diversity measure among generated outputs, so higher values indicate greater perceptual variation when image fidelity is maintained.

1.4.2 Baselines for Comparison

To contextualize the performance of the proposed few-shot image generation model, FeatureSAGE is compared against two editing-based few-shot image generation baselines:

- **AGE (?)**: Leverages latent-space attribute editing on pretrained GANs to produce novel-class images without retraining by disentangling class-relevant and category-irrelevant features.
- **SAGE (?)**: Builds on AGE by aggregating support images to create stable latent embeddings and injecting class-consistent diversity through attribute mixing.

1.5 Expected Outcome

Based on the stated objectives, FeatureSAGE is expected to improve the SAGE pipeline by replacing the pSp-centered inversion stage with the StyleFeatureEditor inverter and Feature Editor. The expected improvement is not a change in the underlying StyleGAN2 generator or in the attribute group editing objective; rather, it is an improvement in how input images are represented before editing. By extracting both W^+ latent codes and intermediate feature tensors, the proposed framework is expected to preserve more local detail while retaining the ability to apply category-irrelevant edits.

The expected outcomes are evaluated on the FUNIT Animal Faces and 102 Flowers datasets under the same one-shot protocol used for the baseline methods. FID is used to measure how closely generated images align with real-image distributions, while LPIPS is used to examine perceptual variation among generated outputs. Within this evaluation design, the expected measurable outcomes are:

1. **Lower FID**, indicating improved visual realism and stronger distributional alignment between real and synthetic images.
2. **Comparable or higher LPIPS as a diversity measure**, indicating that improved fidelity does not eliminate perceptual variation among generated samples.

Together, these outcomes test whether the FeatureSAGE modifications improve few-shot image generation while preserving the edit-based structure of the original SAGE framework.